

From Sparse Selection to Physical Savings: A Runtime Contract for Persistent Native Memory

PRA Research Program

September 2026

Abstract

Logical context sparsity is only an opportunity bound. A runtime may select a small history while rebuilding K/V, copying selected tensors, changing source positions, transferring bytes, constructing K/V-sized temporaries, or inducing additional model calls. We define a vector-valued realization contract for persistent native memory that qualifies semantic same-subset correctness, original-position consumption, resident-state reuse, physical movement, lifecycle, and end-to-end workload economics independently.

The corrected implementations realize disjoint resident K/V through native views or scheduler-owned pages in bounded HF, MLX, SGLang, llama.cpp, vLLM-Metal, and vLLM-CUDA scopes. Matched-consumer probes report zero retained- history re-encoding and zero selected-K/V packing while separately exposing host/device transfer, offload, and bounded attention scratch. The correction ledger shows why these fields cannot be collapsed into “zero-copy”: earlier paths reconstructed caches, used compact rather than original positions, performed hidden casts or copies, and created K/V-sized temporaries.

With frozen RAG selections on Qwen3-4B MLX, cold native execution is 3.2% slower than selected text, but warm repeated selection reduces mean TTFT from 2.640 to 0.166 seconds and total latency from 3.031 to 0.638 seconds. Ordinary exact prefix-cache reuse remains a required baseline; PRA’s distinct advantage is changing or detached addressable evidence. A frozen coding-agent plan omits about 40% of original resident history on multiple engines, yet its economic outcome ranges from preserved call count to reduced benefit or net workload loss as the trajectory changes. Receipt-bearing transfers are explicitly hybrid transformed-context evidence, not strict resident-subset reuse.

The resulting measurement contract establishes two boundaries: logical selection does not imply physical saving, and physical omission does not imply end-to-end workload saving.

1 Introduction

PRA changes what a model must keep logically available. A long-lived resource may remain addressable through a compact routing representation while only a small selected part of its native K/V participates in one attention operation. That distinction is algorithmically useful, but it leaves a systems question unanswered. A selected set can be small while the runtime still stores every detail tensor in HBM, launches many tiny gathers, copies the same bytes repeatedly, constructs temporary concatenations, or serializes requests around Python control flow. In that case sparsity exists in a trace but not in the resource bill.

This paper asks how to turn PRA into a portable inference primitive before a serving engine becomes semantically aware of PRA memory. The immediate task is not to design a new scheduler. It is to define an execution boundary that can be measured, optimized, and later replaced without changing the meaning of a selected reference.

The runtime composes previously defined model, resource, and session interfaces, but this paper evaluates only the native realization boundary. Policy, transport, storage, and tool surfaces remain

separate contracts; their product details are documented in the appendix and companion papers rather than treated as additional systems claims here.

The paper is organized around four independent gates:

- **Semantic gate.** The engine must consume the same selected state as an identical-subset reference from the same resident source.
- **Position gate.** Sparse records retain original source positions; compact selected width is never substituted for the source position base.
- **Physical mechanism gate.** Re-encoding, selected-K/V copy, host–device transfer, consumer temporaries, ownership, and lifecycle are reported independently.
- **Workload gate.** TTFT, total latency, calls, completion work, and task outcome determine whether physical omission becomes an economic saving.

The companion Paper 8.5 owns engine-independent agent-memory discovery, including record exclusion, causal policy, oracle add-back, and logical task efficiency. Paper 4.5 imports a frozen, hash-bound **AgentMemoryPlan**; it does not tune or promote the selector. Full history and 100% retention remain correctness controls before any reduced plan is interpreted.

More precisely, Paper 8.5 decides which logical records or observation spans remain visible. Paper 4.5 consumes a frozen, hashable **AgentMemoryPlan** and tests whether an engine realizes that identical plan from resident K/V with original positions, no selected-history re-encoding or physical copy, safe lifecycle, and separately reported temporary and runtime costs. Engine outcomes do not retune the selector.

The unified runtime, gateway, authenticated resource lifecycle, typed tool and result surfaces, and engine-specific adapters are the implementation needed to test those claims. They are not additional evidence categories. In particular, this paper treats correction of an overstated result as evidence about the measurement contract: cache identity, residence, source position, physical copy, and realized retention must be observed independently.

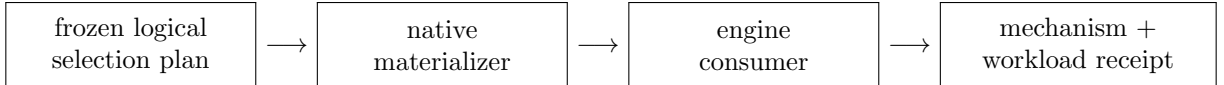


Figure 1: Paper boundary. Policy chooses stable records and spans; the runtime must realize that frozen plan and expose the physical work actually performed.

2 Accounting Model

For L decoder layers, T source tokens, H_{kv} physical K/V heads, head width D , and b bytes per scalar, complete detail storage is

$$B_{\text{detail}} = 2|L_{KV}|H_{kv}TD b.$$

If request-time selection admits S tokens to $|L_C|$ consuming layers, the active payload opportunity is

$$B_{\text{active}} = 2|L_C|H_{kv}SD b.$$

The ratio $B_{\text{active}}/B_{\text{detail}}$ is not peak HBM or runtime. Weights, local K/V, allocator state, routing indices, transfer staging, score buffers, and temporary packing remain outside it.

Quantity	Interpretation
Full/selected logical tokens	Policy opportunity before engine realization.
Visible prompt tokens	Text actually submitted through the ordinary prompt path.
Resident/selected source K/V	Canonical stored extent and the intervals read by this request.
Original-history re-encoding	Old selected tokens evaluated through the model again.
Synthetic-history encoding	Post-cache summaries, tombstones, or receipts evaluated as new tokens.
Partial-old-history encoding	Previously seen but uncached tails evaluated again, for example an incomplete page.
Selected-K/V copy and H2D	Explicit packing/copy and residency transfer, reported separately.
Consumer temporary bytes	Peak request-local score, reduction, concatenation, and allocator state.
Workload	Requests, TTFT, total latency, completion tokens, peak memory, and task outcome.

Table 1: Independent accounting quantities. No single “reuse” or “zero-copy” label substitutes for these fields.

Evidence is reported in four classes. A *semantic gate* verifies that the selected state produces the intended same-subset result. A *mechanism gate* observes positions, historical evaluation, copying, and ownership. A *physical-efficiency gate* measures resident bytes, transfer, temporaries, and peak memory. A *workload gate* measures requests, TTFT, total latency, completion work, and task outcome. An engine can pass the first two and still fail the latter two; the vector must not be collapsed into “PRA supported.”

The accounting receipt is request-scoped and stage-labelled. Bytes copied to bootstrap a canonical source are different from bytes copied for every selection. Similarly, one-time ingest encoding is different from re-encoding already resident history. Peak allocation is reported separately from cumulative traffic, because a small reusable scratch buffer and a newly packed K/V tensor can have similar peaks but radically different repeated-request costs.

3 Native Realization Contract

The core contract is `live-kv-original-position-lifecycle-v1`. A request identifies the exact source cache, runtime/model/tokenizer revision, stable selected records and spans, original logical positions, session and tenant scope, and a plan digest. A same-subset comparator consumes identical K/V values at identical positions. Ownership remains with the canonical source while requests borrow intervals; finish, cancellation, error, eviction, offload/restore, termination, and stale generation rejection are independently tested.

Qualification is vector-valued:

$$G = (S, P, C, L, E),$$

where S is same-state semantic correctness, P is original-position sparse correctness, C is copy/re-encoding behavior, L is lifecycle correctness, and E is end-to-end workload qualification. Passing one coordinate does not promote another.

After one source bootstrap, strict resident-subset execution permits only genuinely new history delta, the current suffix, and generated tokens. It requires

$$R_{\text{orig}} = R_{\text{synthetic}} = R_{\text{partial}} = 0, \quad C_{\text{selectedKV}} = 0.$$

Ingest-time compaction may create reusable K/V before source admission. Replacing already resident records with new receipt text is instead a hybrid transformed-context treatment and cannot qualify this invariant.

3.1 Position and composition semantics

The plan carries source/original positions, while an implementation may also have chunk-local and request-packed coordinates. These are not interchangeable. For rotary models, a cached key associated with logical position p contains the corresponding rotation (schematically $K_{\text{post}}(p) = R_p K_{\text{pre}}$). Relabelling an already rotated key or compacting disjoint records into positions $0, \dots, S - 1$ changes the attention state. The same-subset comparator therefore uses the identical selected K/V and original positions; it does not compare sparse original-position execution with a newly packed text prompt and call any difference an implementation bug.

Composition is separately declared. Independently encoding chunks at local positions can be a valid alternative representation, but it is a different treatment from globally encoding the concatenated document. The engine must name which representation it consumes and qualify quality under that representation rather than infer equivalence from token identity.

3.2 Ownership and lifecycle

The canonical source owns resident K/V; requests borrow immutable intervals or pages. A lifecycle gate covers concurrent session isolation, cancellation, normal termination, stale-fork rejection, eviction, offload/restore, and final cleanup. Releasing one borrower must not free state still referenced by another, while eviction must either wait for borrowers or move through a generation-checked handoff. These tests are mechanism evidence, not throughput benchmarks, but they are required before an aliasing implementation can be called reusable.

Paper 8.5 supplies the following frozen handoff:

```
AgentMemoryPlan
  selected stable record IDs and spans
  original logical positions
  materialization/compaction stage
  source and plan digests
```

The engine realizes this plan; it does not infer task semantics or retune the policy.

4 Runtime Architecture

The contract is implemented by a narrow set of components. The application or Paper 8.5 policy emits a logical plan; a resource registry resolves stable identities to a canonical source generation; a materializer validates spans and positions; an engine adapter creates non-owning views or scheduler aliases; a lifecycle manager owns eviction and restoration; and telemetry records the work performed. Tool schemas, generic gateway UX, and policy tuning are not part of this core path.

Component	Authority	Required receipt
Selection plan	stable record IDs, spans, positions, digest	selected logical extent and materialization stage
Resource registry	tenant/session/model/source generation	exact source-cache identity and residency state
Materializer	interval validation and consumer descriptor	rejected spans, old/synthetic/partial encoding counts
Engine adapter	attention consumer and native alias geometry	K/V copy, transfer, temporary, and position mode
Lifecycle manager	borrow, release, eviction, restore, cleanup	ownership generation and terminal-state outcome
Workload harness	prefix/full controls and task execution	TTFT, latency, calls, completion work, peak, and quality

Table 2: Minimal architecture supporting the native realization contract.

A plan digest binds policy output to runtime evidence. If an adapter changes the selected intervals, compacts positions, inserts receipt text, or falls back to prompt replay, it must emit a distinct realization digest and accounting class. This makes silent degradation observable and permits the same frozen logical treatment to be compared across engines.

The ordinary prefix cache remains available beside addressable resources. Local request deltas should continue through the engine’s normal incremental path; persistent source records remain separately addressable. PRA is useful when the next request needs a disjoint or changing subset that cannot be represented as one unchanged prefix. It should not replace an exact-prefix hit with a more expensive sparse path.

5 Corrected Engine Matrix

Engine	Model/backend scope	<i>S</i>	<i>P</i>	<i>C</i>	<i>L</i>	<i>E</i>	Primary remaining limitation
HF	Qwen/CUDA, batch 1	pass	pass	pass	pass	open	Autonomous frozen-plan workload not rerun after the final contract.
MLX	Direct MLX, batch 1	pass	pass	pass	pass	diag.	Topology/head-dimension bounds; current agent transfer is hybrid.
SGLang-MLX	Direct MLX row consumer	pass	pass	pass	pass	diag.	Rows are evaluated independently; current transfer is hybrid and loses workload.
llama.cpp	Resident slot/sequence ranges	pass	pass	pass	pass	1 task	Endpoint temporary allocation remains incomplete; current transfer is hybrid.
vLLM-Metal	Bounded page geometry	pass	pass	pass	24/24	open	No frozen autonomous reduced-plan workload row.
vLLM CUDA	V1, complete homogeneous pages	pass	pass	bounded	pass	open	Frozen-plan bridge still needs an autonomous no-copy workload closure.

Table 3: Vector-valued qualification $G = (S, P, C, L, E)$: same-state semantics, original positions, copy/re-encoding, lifecycle, and autonomous workload. “Pass” is limited to the declared scope; “diag.” and “open” are not inferred from mechanism success.

The primary correction ledger is consolidated in Appendix A. Earlier rows that reconstructed caches, used compact selected positions, performed hidden casts/copies, or created K/V-sized temporaries remain debugging evidence only.

6 Results

6.1 Warm repeated selection produces a real native-memory benefit

The cleanest realization experiment freezes 50 Qwen3-4B MLX RAG selections at 2K context. Selected text and native memory match every output and quality metric. Cold native execution costs 3.121 versus 3.024 seconds, a 3.2% regression from native encoding. Once the identical selection is resident, mean TTFT falls from 2.640 to 0.166 seconds and total latency from 3.031 to 0.638 seconds; throughput rises from 10.56 to 50.21 tokens/s. At 4K, cold native is 3.3% slower while warm total latency is 0.799 versus 6.121 seconds. Qwen3-8B repeats the pattern: 1.8% cold regression, then an 82.1% warm total-latency reduction. These are repeated-selection results on one M4 host, not general serving speedups.

Model / context	Cold selected	Cold native	Warm selected	Warm native
Qwen3-4B / 2K total (s)	3.024	3.121	3.031	0.638
Qwen3-4B / 2K TTFT (s)	—	—	2.640	0.166
Qwen3-4B / 4K total (s)	—	+3.3%	6.121	0.799
Qwen3-8B / 2K total	—	+1.8%	baseline	−82.1%

Table 4: Repeated-selection realization summary. Dashes indicate that the main claim uses the reported relative effect rather than reconstructing an unreported absolute cell.

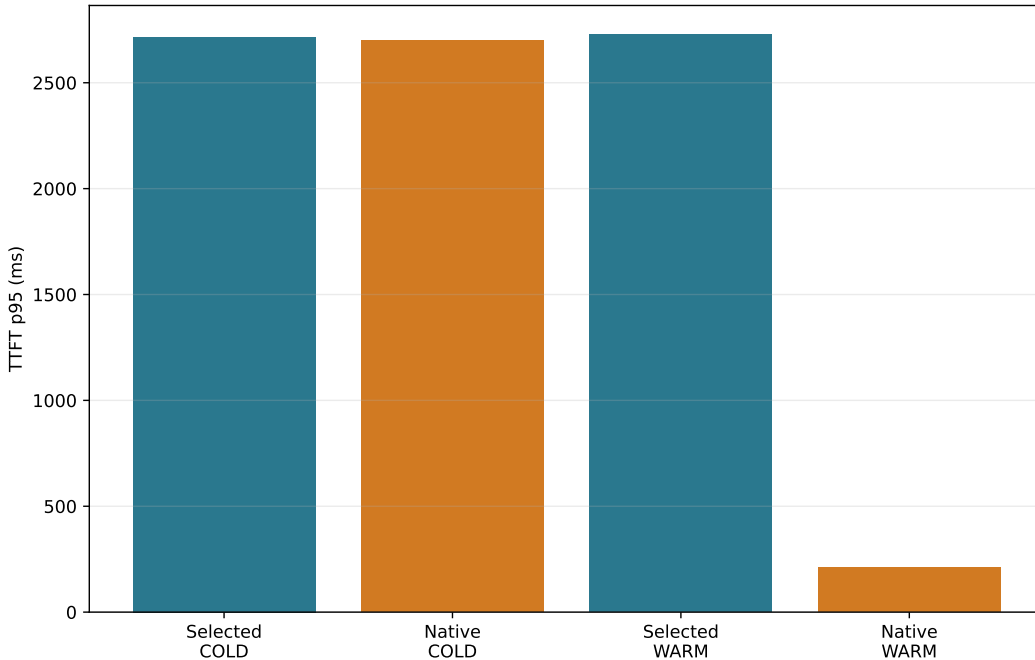


Figure 2: Qwen3-4B, 2K, 50-query mechanism cohort. Cold native realization has no material TTFT advantage; warm residence removes repeated selected-text prefill. Values are host- and cohort-specific.

6.2 Exact prefix caching is a necessary baseline

Warm native memory must not be compared only with fresh text prefill. The required ladder is fresh selected text, exact text-prefix hit, persistent native selection, and changing-selection partial native reuse. An autonomous llama.cpp control illustrates the strength of the baseline: at 100% retention,

ordinary live-prefix continuation matches cold-prefill behavior while reducing single-task wall time by $27.85\times$. This is a prefix-cache result, not a PRA speedup. PRA’s distinct systems opportunity begins when selected evidence is detached, disjoint, or changes across requests.

6.3 Complete retention must remain on the native dense path

The corrected HF dispatcher no longer sends PRA-100 through the disjoint online-softmax consumer. It shallow-forks only request-local cache descriptors while every key/value tensor continues to alias the pinned canonical source; request growth replaces private descriptors without mutating that source. Transformers therefore executes its ordinary dense attention and last-logit projection geometry. A real Qwen2.5-0.5B FP16 control with 2,713 resident source tokens is token exact and has maximum logit delta 0.0 against two independent ordinary-prefix executions and after lossless restore. It reports zero selected-history re-encoding, selected-K/V copy, interval packing, or consumer K/V temporary. This is a bounded correctness result, not an autonomous workload or latency claim. Earlier PRA-100 probes that forced a complete contiguous prefix through a sparse consumer remain useful numerical diagnostics but are not admissible full-retention implementations. The machine-readable receipt is `shared/results/paper4_5_runtime_productization/coding_agents/engine_gates/hf_qwen25_05b_full_native_dense_fix_20260919.json`.

6.4 Changing selection changes composition semantics

Persistent-corpus MLX reuses independently encoded 256-token chunks across 50 changing questions. It reuses 22.0% of chunk lookups and reduces cumulative request time by 12.8%, but official score changes from 0.440 to 0.380. The cause is representational: chunk-local post-RoPE positions differ from the single global position sequence of packed text. This is not cache corruption. It establishes

same selected evidence $\not\Rightarrow$ same hidden-state semantics.

Original, chunk-local, and request-packed positions are therefore explicit contract fields rather than implementation details.

6.5 The numerical consumer path is part of the treatment

On the corrected SGLang coding-agent control, fresh full prefill and live-prefix PRA-100 receive identical input token IDs, resolve the same task in seven calls, and submit a byte-identical patch, yet their assistant actions first diverge at request three. A selection-neutral live-prefix control then matches PRA-100 on all seven requests, responses, and commands. Thus

same token IDs $\not\Rightarrow$ same numerical consumer path.

Reduced-policy comparisons must use a matched consumer; fresh-prefill identity cannot be silently required of an incremental path.

6.6 Frozen agent-plan transfer separates omission from economics

Paper 8.5 supplies one Prompt Pinned plan on Scikit-learn-14496. The current transfers are hybrid because they create model-visible closure receipts after the source cache exists; they qualify original-K/V selection but not the strict resident-subset invariant.

Engine	PRA-100	Reduced	Calls F/R	Logical/KV omission	Hist. re-enc.	K/V copy	Paired workload	Status
llama.cpp	pass	solve	8/8	40.23/40.23%	0 orig.; receipts n/a	0 selected	calls preserved	hybrid; timing open
MLX	pass	solve	9/12	39.45/40.04%	1,056 synth.	0 selected	+15.23% saving	hybrid
SGLang-MLX	pass	solve	7/15	39.45/37.21%	1,344 synth.	0 selected	−33.10% saving	hybrid
HF CUDA	mech.	open	—	—	—	—	open	autonomous gate open
vLLM CUDA	mech.	open	—	—	—	—	open	autonomous no-copy gate open

Table 5: Frozen-plan workload economics. F/R denotes matched Full/reduced calls. Positive paired workload is saving; negative is regression. Receipt rows are hybrid transformed context and are not pooled with strict resident- subset execution.

These rows establish the second systems boundary:

physical omission $\nrightarrow$ end-to-end workload saving.

Selection-only reruns with zero synthetic history are required before the plan can enter the strict native class.

7 Prefix Cache and PRA Are Complementary

Exact prefix caching is the correct default when the next request extends one unchanged token prefix. It preserves the engine’s native consumer path and can reuse all prefix K/V with minimal planning. PRA addresses a different shape: the request needs a changing, detached, or disjoint subset of already resident records. A runtime should choose between the two using observable request geometry, not route every request through sparse selection.

Request shape	Preferred path	Required baseline	Main risk
unchanged prefix plus suffix	exact prefix cache	live-prefix continuation	replacing a near-perfect hit with planning/copy overhead
same frozen detached evidence	persistent native selection	warm selected-text and prefix hit where representable	residence cost and consumer temporaries
changing disjoint records	partial native PRA	fresh selected text plus matched consumer	position/composition mismatch and churn
post-cache summary or receipt	hybrid transformed context	selected K/V plus charged synthetic encoding	describing new text as cache reuse
uncached or partial source tail	mixed/fallback path	explicit partial-history accounting	hiding old-token evaluation in a nominal hit

Table 6: Dispatch rule and comparator ladder by request geometry.

This division also clarifies the value proposition. PRA does not need to beat prefix caching on unchanged prefixes; it must outperform rebuilding or copying the selected state when ordinary prefix identity no longer applies. The telemetry receipt therefore records exact-prefix reused tokens and addressable resident intervals separately. A combined runtime can keep the local agent delta on the prefix path while borrowing older resource intervals through PRA, without re-encoding either class.

8 Qualification and Promotion Procedure

Qualification proceeds from semantics to workload; later measurements cannot repair an earlier failure. First, a deterministic bounded geometry compares native sparse execution against a same-subset reference with identical K/V and positions. Second, instrumented counters require zero original-history

evaluation and separately expose synthetic or partial-history work, selected K/V copies, transfers, and consumer temporaries. Third, lifecycle tests cover concurrent borrowers and every terminal path. Fourth, cold, warm, and changing-selection cohorts measure physical and latency effects against both fresh prefill and exact prefix caching. Finally, an autonomous workload consumes a frozen policy plan without engine-specific retuning.

Gate	Required comparator	Promotion evidence
Semantic	identical selected K/V and original positions	bounded output/logit agreement under declared dtype and consumer path
Mechanism	instrumented resident source	zero old-history evaluation; copy, transfer, temporary, and lifecycle fields reported
Physical	cold/warm/changing selection	resident and request-local bytes plus peak and cumulative traffic
Workload	prefix cache and fresh prefill	task outcome, requests, TTFT, latency, completion work, and paired cumulative cost

Table 7: Promotion ladder. A row is not inferred from success in a later row.

This ordering prevents three common category errors. Matching output alone does not prove that historical K/V was reused. Zero selected-K/V copy does not prove low temporary memory or low transfer. A fast request does not prove an end-to-end saving if the model takes more requests to solve the task.

9 Scope and Remaining Gates

The corrected adapters establish bounded mechanisms, not universal engine support. HF remains qualified only for declared Qwen/CUDA batch-one geometry; MLX and SGLang retain batch, topology, and head-dimension restrictions; llama.cpp still lacks complete endpoint temporary-allocation visibility; and vLLM CUDA remains limited to in-process V1, complete pages, and a homogeneous K/V group. Partial pages, hybrid groups, multiprocess scheduling, broader models, and production concurrency require separate gates.

The warm RAG result is a repeated-selection mechanism cohort on one Apple host. It demonstrates that correct residence can remove repeated prefill, but it does not establish portable serving speedups. The changing-selection experiment also shows why globally packed text cannot be treated as the universal semantic reference for independently encoded chunks.

The current agent transfer is one frozen task-plan identity and is hybrid because closure receipts are encoded after caching. Its value is diagnostic: similar original-K/V omission leads to preserved, reduced, or negative workload benefit across consumers. The strict next experiment removes synthetic history, holds the selected records fixed, and repeats the paired autonomous gate with task-clustered replication. HF and vLLM CUDA have bounded mechanism evidence but no admissible post-contract autonomous reduced-plan row; their workload coordinate E therefore remains explicitly open.

10 Conclusion

Sparse logical selection is an opportunity bound, not a systems result. Native reuse requires source identity, original positions, same-subset correctness, zero hidden historical evaluation, scoped K/V movement, bounded consumer state, and safe ownership. Correct persistent native memory can deliver large warm repeated-selection gains, while exact prefix caching remains a formidable baseline. Changing selections introduce composition semantics that ordinary prefix reuse does not solve. Finally, an agent

can react to physically correct omission by taking more calls, erasing or reversing the saving. The runtime contract therefore ends at vector-valued evidence, not a single reuse flag.

A Correction Ledger

The runtime contract was tightened after agent-history experiments exposed several ways to overstate native reuse. Table 8 records the corrections that invalidate earlier economic claims while preserving their value as debugging evidence. The complete chronology and raw tables remain in the technical-report edition.

Earlier treatment	Why it is invalid	Current requirement
Reconstructed cache	Fresh prefill is not the same resident state.	Fork or alias the canonical live cache; report old-token evaluation.
Compact position base	Disjoint records changed rotary/causal coordinates.	Preserve original source positions in the plan and consumer.
Hidden cast or pack	Selected K/V was copied while reported as reuse.	Report selected-K/V copy, dtype conversion, and H2D separately.
K/V-sized temporary	Selection was view-based but attention concatenated resident K/V.	Report consumer peak and cumulative temporaries independently.
Retention ceiling	Whole records rounded below a nominal percentage.	Use the authoritative tokenizer and fail closed below the declared floor.
Mixed revisions	Mechanism and workload rows combined different packages or patches.	Bind model, tokenizer, engine, PRA revision, source, and plan digests.
Post-cache receipt	New summary text was described as resident-subset reuse.	Classify and charge transformed context separately from strict native reuse.

Table 8: Authoritative correction ledger for publication claims.

B Bounded Engine Scopes

Engine qualification is deliberately scoped rather than binary. HF covers Qwen2/Qwen3 full-attention models at batch one; MLX and SGLang-MLX cover their declared Metal topology; llama.cpp covers resident sequence/range attachment; and vLLM CUDA covers in-process V1 with complete homogeneous pages. These rows do not imply partial-page, hybrid-group, multiprocess, distributed, or production-concurrency support. A mechanism-qualified row retains $E = \text{open}$ until a matched autonomous workload reports task outcome, calls, completion work, prefix-cache status, physical traffic, and repeated latency.

The frozen agent-policy handoff contains the policy ID, plan and source-prefix digests, selected stable record IDs, spans, original positions, and materialization class. Any runtime that changes one of those fields creates a new realization row. Selection-only resident subsets and receipt-bearing hybrid transformed context are never pooled.

C Reproducibility Map

The implementation and evidence are rooted at the following repository paths:

- `src/prahf/hf_live_kv.py`, `src/pramlx/native.py`, `src/prasglang/`, and `src/pravllm/` implement the bounded native consumers and ownership contracts.
- `experiments/paper4_5_agent/` contains the matched FULL, PRA-100, frozen-plan, lifecycle, and autonomous workload runners.
- `docs/papers/shared/results/paper4_5_runtime_productization/agent_memory_plans/` contains hash-bound Paper 8.5 plans, engine receipts, and autonomous outcomes.
- `docs/papers/shared/results/paper4_5_runtime_productization/rag/` contains frozen-selection cold/warm realization evidence.

Every headline value in the publication edition maps to a machine-readable artifact under these roots. The technical-report edition retains the full implementation guide, gateway and storage studies, historical matrices, and superseded diagnostic tables.